



SwaggerTM

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 2	Tema: Documentando APIs
	Semana 6	Tutor: Ing. Victoria Castro

Objetivo:

Comprender la arquitectura de OpenAPI y ejecutar la documentación automatizada de servicios REST.

Introducción:

El Ecosistema OpenAPI

Para entender Swagger, primero debemos definir la **Especificación OpenAPI (OAS)**. Esta es una descripción estándar de interfaz para APIs HTTP que permite a los humanos y a las computadoras descubrir y comprender las capacidades del servicio sin acceso al código fuente.

Sección 1:

Diferencia entre Swagger y OpenAPI

- **OpenAPI:** Es el estándar (la regla). Es un documento escrito en JSON o YAML que describe los endpoints, parámetros y respuestas.
- **Swagger:** Es el herramental (la implementación). Es el software que lee ese archivo OpenAPI para generar la interfaz visual (Swagger UI), el editor o el cliente de código.
-

Arquitectura de Introspección en Spring Boot

En el ecosistema Java, no escribimos el archivo YAML a mano. Utilizamos la librería **Springdoc-OpenAPI**. El proceso técnico funciona así:

1. **Reflection & Introspection:** Al iniciar la aplicación, la librería escanea el Application Context de Spring.
2. **Mapping:** Identifica las anotaciones de Spring Web (@RestController, @RequestMapping).
3. **Metadatos:** Extrae tipos de datos, validaciones (JSR-303) y estructuras de retorno.
4. **Exposición:** Genera dinámicamente el contrato en la ruta /v3/api-docs.

Sección 2:

Instalación y Configuración del Entorno

Para Spring Boot 3 (basado en Jakarta EE), la dependencia estándar es el *starter* de Springdoc.

Dependencia Maven

Añade lo siguiente en tu pom.xml:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>
```

Configuración en application.properties

Aunque funciona *out-of-the-box*, un arquitecto debe tener control sobre las rutas:

Ruta para acceder a la interfaz gráfica

springdoc.swagger-ui.path=/docs

Ruta donde se sirve el JSON técnico (útil para herramientas externas)

springdoc.api-docs.path=/api-docs

Ordenar los endpoints por método HTTP en la UI

springdoc.swagger-ui.operationsSorter=method

Sección 3:

Implementación: Documentación de Código

La documentación se divide en tres niveles: la API global, los controladores (operaciones) y los modelos (esquemas).

Para definir el título, versión y servidores, creamos una clase de configuración:

@Configuration

public class OpenApiConfig {

 @Bean

 public OpenAPI customOpenAPI() {

 return new OpenAPI()

 .info(new Info()

 .title("API de Inventario Global")

 .version("1.0")

 .description("Sistema centralizado para la gestión de productos y stock.")

 .contact(new Contact().name("Ingeniería de Software").email("dev@u.edu"));

 }

}

Documentación de Endpoints (Controladores)

Usamos anotaciones para describir la intención de cada método:

- @Tag: Agrupa operaciones bajo un mismo nombre.
- @Operation: Describe qué hace el endpoint.
- @ApiResponse: Documenta los posibles resultados (éxito y errores).

```
@Tag(name = "Productos", description = "Gestión del catálogo de productos")
```

```
@RestController
```

```
@RequestMapping("/api/v1/productos")
```

```
public class ProductoController {
```

```
    @Operation(summary = "Buscar por ID", description = "Obtiene los detalles de un  
    producto específico")
```

```
    @ApiResponses(value = {
```

```
        @ApiResponse(responseCode = "200", description = "Producto encontrado"),
```

```
        @ApiResponse(responseCode = "404", description = "El ID proporcionado no existe")
```

```
    })
```

```
    @GetMapping("/{id}")
```

```
    public Producto getById(@PathVariable Long id) { ... }
```

```
}
```

Documentación de Modelos (Schemas)

Es vital que el consumidor de la API conozca las restricciones de los datos.

```
@Schema(description = "Entidad que representa un artículo en el sistema")
```

```
public class ProductoDTO {
```

```
    @Schema(example = "101", description = "Identificador único auto-generado")
```

```
    private Long id;
```

```
    @Schema(required = true, example = "Laptop Gamer", minLength = 3)
```

```
    private String nombre;
```

```
}
```

Sección 4:

Mejores Prácticas y Seguridad

- **Validación Automática:** Si usas `@NotNull` o `@Size` de Hibernate Validator, Swagger las detectará y las marcará como obligatorias en la UI automáticamente.
- **Producción vs Desarrollo:** Por seguridad, se recomienda **desactivar** Swagger en entornos de producción mediante perfiles de Spring: `springdoc.api-docs.enabled=false`
- **Contratos Primero:** Aunque aquí generamos documentación desde el código, en proyectos grandes se suele diseñar el YAML primero y luego generar el código.

Bibliografía y Recursos de Profundización

Para ampliar los conocimientos de este material, se sugiere consultar:

1. OpenAPI Specification (OAI): <https://spec.openapis.org/oas/v3.1.0> - El estándar oficial.
2. Springdoc.org: <https://springdoc.org/> - Documentación técnica de la librería utilizada.
3. Swagger Documentation Hub: <https://swagger.io/docs/> - Guías sobre el ecosistema de herramientas.

